SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ITA0329– Mobile Computing Assignment 5 (CO5)

NAME: Gayathri. P

REG NO:192424324

PART A – DESIGN, ANALYSIS & ARCHITECTURAL REASONING

TASK A1 – Design a Constraint-Aware Intent & Service Architecture

Question. A healthcare monitoring application must periodically collect sensor readings, upload them to a backend when connectivity is available, and notify the user of critical events. Background work must respect battery constraints, Android background-execution limits and data privacy. Model each data-collection / upload task as a Service or WorkManager worker, each inter-component communication as an Intent, and each security / battery / connectivity constraint as a pruning / feasibility condition.

A1.1 – Model the Problem

Question. Identify input, output, the decision made at each stage, the constraints and the goal condition.

Answer.

Input

Input class	Concrete items
Sensor events	Heart rate, SpO_2, temperature, accelerometer (fall detection) delivered by Sensor Manager or a BLE wearable callback
Network state	Connected / not connected, transport (Wi-Fi, cellular), measured quality (good / poor / none), metered flag, from Connectivity Manager. Network Callback
User preferences	Upload on Wi-Fi only, quiet hours, notification channel settings, consent flags
Device state	Battery percentage, charging, Doze / App-Standby bucket, Battery Saver
Security state	Runtime permissions (BODY_SENSORS, POST_NOTIFICATIONS), session token, Keystore key availability

Output

Output class	Concrete items
Uploaded records	Encrypted batches acknowledged by the backend (idempotent, each with a record id)
Notifications	Local critical-event alert (e.g. SpO_2 below 88 %), "sync completed / delayed" status
Local residue	Encrypted records still waiting in the local store (the *backlog*)

Decision made at each stage

Stage	Decision
Collection	Sample now (foreground service) or batch via scheduled sampling? At which rate?
Classification	Is this reading *critical* (P0), *high* (P1), *normal* (P2) or *low* (P3)?

Stage	Decision
Scheduling	For the next work item: execute immediately (expedited), enqueue as constrained WorkManager work, defer, or cancel?
Upload	Which batch, in what order, how large a chunk, with what back-off?
Notification	Notify now (heads-up), notify silently, or escalate through an ordered broadcast?

Constraints

ID	Constraint	Source
C1	Runtime permission must be granted and session valid	Android permission model
C2	Sensitive data must be encrypted (AES-256-GCM, key in Android Keystore) *before* it is written to disk or sent	Privacy / HIPAA-style rule
C3	Battery $\geq \theta$(priority) or device is charging	Battery constraint
C4	Doze / App Standby: non-critical work must not run in maintenance-free windows	Android background limits
C5	Network available and of sufficient quality for the payload size	Connectivity
C6	Background service start restrictions (Android 8+/12+): long work must be a foreground service of declared type or WorkManager	Platform policy
C7	Responsiveness: no work on the main thread, UI thread stays under 16 ms per frame	Usability

Goal condition. The system reaches a *goal state* when, for every record r that has entered the system:

1. r is stored encrypted (C2), **and**
2. r is either acknowledged by the backend exactly once, **or** r remains encrypted locally with a valid pending work request (nothing is lost, nothing is leaked), **and**
3. every critical event has produced a user-visible notification within its deadline, **and**
4. after logout or permission revocation no background work for that user is running or scheduled and local plaintext is zero.

Component mapping (Service / WorkManager and Intents).

Task	Android component	Why this component
Continuous sensor collection	SensorCollectionService – foreground service, type health, with an ongoing notification	Needs long-running access to sensors; background services are not allowed to start this from the background
Periodic batch upload	UploadWorker (CoroutineWorker) with constraints NetworkType.CONNECTED, RequiresBatteryNotLow, exponential back-off	Deferrable, guaranteed, survives process death and reboot, honours Doze
Critical event handling	CriticalAlertWorker – *expedited* work (setExpedited)	Must run promptly even under quota/Doze rules
User-facing alert	NotificationManager + PendingIntent (immutable)	Visible alert; tap opens the patient screen
Component messaging	See Intent table below	Explicit, implicit and ordered Intents

Communication	Intent kind	Detail
Activity → start collection	**Explicit** Intent to SensorCollectionService	Same package, no exported component, no interception possible
Service → UploadWorker hand-off	Work request Data + explicit local broadcast (setPackage)	Keeps the payload inside the app
System → app: battery low / power connected	**Implicit** (system) broadcast registered at runtime	Feeds environment vector E
Critical event fan-out	**Ordered** broadcast, receivers with priority: (1) in-app alert receiver, (2) paired wearable bridge, (3) logger	Higher-priority receiver can mark the alert handled and *abort* the lower ones
Notification tap	Explicit PendingIntent with FLAG_IMMUTABLE	Prevents tampering by other apps

A1.2 – Define the Runtime State

Question. Analyse what information must be maintained while a background task is partially executed or waiting for network.

Answer. Android may kill the process at any moment, so the state of a partially executed task must live in durable storage (an encrypted Room database), not in memory. The runtime state is the tuple

σ = (Q, W, E, K, L, C)

Component	Content	Why it must be kept
Q – queue	For each item: id, type (collect / upload / alert), priority P0–P3, size, encrypted?, retries, createdAt, deadline (TTL), notBefore	Needed to select the next item and to decide deferral vs cancellation
W – active workers	worker id, item id, start time, bytes sent	Enforces the concurrency cap and allows cancel on logout
E – environment	battery %, charging, network class, Doze flag, permission flags, session valid	Evaluated at every decision point; snapshot is logged for diagnosis
K – crypto session	Keystore alias, current key version, GCM IV counter	A repeated IV breaks AES-GCM security; key rotation must survive restart
L – ledger	Set of record ids acknowledged by the server, with server timestamp	Idempotency: a retry after a lost ACK must not duplicate a record
C – checkpoint	For a partly uploaded batch: chunk index / byte offset / upload session URI	Resume rather than restart; avoids re-sending data and wasting energy

What would go wrong without each part: without *C* every network drop restarts the batch (energy waste); without *L* a lost ACK causes duplicate clinical records; without *K* a reboot may reuse an IV; without *E* snapshots a failure cannot be diagnosed; without *W* logout cannot cancel running uploads.

A1.3 – Define the Pruning / Feasibility Condition

Question. Specify precisely when a scheduled background task can be deferred, cancelled or rejected without violating correctness or security.

Answer. For a work item *w* in environment *E* define the atomic predicates

- PERM(E) = permission granted ∧ session valid
- ENC(w) = w is encrypted (or can be encrypted now with an available key)
- NET(w,E) = network class of E meets need(w) (none < poor < good; need = poor for items ≤ 6 KB, good otherwise)
- BAT(w,E) = battery ≥ θ(priority(w)) ∨ charging, with θ(P0)=0 %, θ(P1)=20 %, θ(P2)=20 %, θ(P3)=30 %
- DOZE(w,E) = ¬Doze(E) ∨ expedited(w)
- ALIVE(w,t) = t − createdAt(w) ≤ TTL(w)

Feasible(w,E,t) = PERM ∧ ENC ∧ NET ∧ BAT ∧ DOZE ∧ ALIVE

The outcome of a failed test depends on *which* predicate failed, because transient and permanent failures have different safe responses:

Failed predicate	Class	Safe action	Justification
NET, BAT, DOZE	Transient	**DEFER** (keep encrypted in Q, set notBefore, re-test on the next state change)	The condition will become true again; no correctness is lost, only latency
ALIVE (non-critical)	Time-bound	**CANCEL** the *delivery attempt*, keep or drop the record according to retention rules	Data stale; sending late wastes energy
ALIVE (critical)	Time-bound but unsafe to drop	**ESCALATE** – raise a local notification, keep the record	A missed clinical alert must never vanish silently
PERM	Permanent until user acts	**CANCEL all** work of that user, **purge** queue of upload intents, keep encrypted data only if policy allows	Without permission any sensor/upload work is a privacy violation

Failed predicate	Class	Safe action	Justification
ENC	Structural	**REJECT** (do not enqueue, do not send)	Plaintext must never leave the device

Soundness argument. A deferred or cancelled item never causes a leak because every path out of the device passes through the single gate Feasible(); and it never causes loss because deferral keeps the encrypted item in Q with a valid pending request. Cancellation is only chosen when delivery is *impossible* (PERM) or *worthless* (stale non-critical), and the critical case is excluded from silent cancellation.

A1.4 – Design Complete Architectural Pseudocode

Question. Write language-independent pseudocode showing selection of next work item, candidate Intent / Service type, feasibility test, enqueue / start, result handling with undo / cancellation, and success / failure condition.

Answer.

```
ALGORITHM BackgroundController(σ)          // σ = (Q, W, E, K, L, C)
CONSTANT  MAX_WORKERS = 3,  MAX_RETRY = 4

ON EVENT e IN { SENSOR_READING, NET_CHANGE, DOZE_CHANGE,
BATTERY_CHANGE,
        PERMISSION_CHANGE, LOGOUT, WORK_RESULT, TIMER_TICK }

  1. UPDATE E from e                       // refresh environment

  2. IF e = LOGOUT OR PERM(E) = FALSE THEN
       FOR EACH w IN W DO  CANCEL(w) ; UNDO(w)      // see UNDO below
       FOR EACH i IN Q DO  REMOVE i FROM Q ; MARK i 'cancelled'
       WIPE session keys from memory
```

```
    RETURN                              // terminal for this user

3. IF e = SENSOR_READING THEN
    r  <- ENCRYPT(e.reading, K)             // C2 enforced at source
    p  <- CLASSIFY(r)                       // P0..P3
    item <- NEW(id, type=upload, p, size(r), encrypted=TRUE, deadline=now+TTL[p])
    ADD item TO Q
    IF p = P0 THEN  RAISE_ALERT(item)       // ordered broadcast + local notification

4. REPEAT
    candidates <- { i IN Q  WHERE  i.notBefore <= now }
    IF candidates = EMPTY OR |W| = MAX_WORKERS THEN BREAK

    i <- SELECT_NEXT(candidates)            // see rule below

    IF NOT ALIVE(i, now) THEN
        IF i.priority = P0 THEN ESCALATE(i) ELSE CANCEL_ITEM(i)
        CONTINUE

    kind <- CHOOSE_COMPONENT(i)             // candidate Intent / Service type
    ok, reason <- FEASIBLE(i, E)

    IF ok THEN
        REMOVE i FROM Q ;  ADD i TO W
        START(kind, i, C[i])                // explicit Intent / enqueue / expedite
    ELSE IF reason IN { NET, BAT, DOZE } THEN
        i.notBefore <- now + BACKOFF(i.retries)  // DEFER; no attempt is made
        LOG(i, E, 'deferred', reason)
    ELSE                                    // ENC or PERM
        REJECT(i) ; LOG(i, E, 'rejected', reason)
  UNTIL no item was started in this pass
```

```
   5. IF e = WORK_RESULT(w, result) THEN
        REMOVE w FROM W
        IF result = SUCCESS(ack) THEN
            ADD w.id TO L ;  DELETE w.records FROM local store ; CLEAR C[w]
        ELSE IF result = RETRYABLE_FAILURE THEN
            w.retries <- w.retries + 1
            IF w.retries > MAX_RETRY THEN  FAIL_ITEM(w) ; UNDO(w)
            ELSE  ADD w TO Q with notBefore = now + BACKOFF(w.retries) ; SAVE C[w]
        ELSE                                  // PERMANENT_FAILURE
            FAIL_ITEM(w) ; UNDO(w)

END ALGORITHM

FUNCTION CHOOSE_COMPONENT(i)
   IF i.type = collect              RETURN ForegroundService (explicit Intent)
   IF i.priority = P0               RETURN ExpeditedWork      (explicit Intent / setExpedited)
   IF i.size <= SMALL AND device charging RETURN OneTimeWork (constraints: connected)
   RETURN PeriodicOrOneTimeWork(constraints: connected, battery-not-low)
END

FUNCTION SELECT_NEXT(C)                 // reasoned selection rule
   RETURN argmax over i IN C of  [ 10*prio(i) + 6*age(i)/TTL(i) + 2/(1 + size(i)/4) ]
END

FUNCTION FEASIBLE(i, E)
   IF NOT PERM(E)                    RETURN (FALSE, PERM)
   IF NOT i.encrypted                RETURN (FALSE, ENC)
   IF NOT NET(i, E)                  RETURN (FALSE, NET)
   IF NOT BAT(i, E)                  RETURN (FALSE, BAT)
   IF Doze(E) AND i.priority <> P0         RETURN (FALSE, DOZE)
```

```
    RETURN (TRUE, OK)
END

FUNCTION UNDO(w)                    // restore previous state after cancel / failure
    ROLLBACK partial upload session on server (abort session id)
    IF records of w were marked 'in-flight' THEN mark them 'pending' again
    RELEASE wake locks, close sockets, delete temporary plaintext buffers
    KEEP records encrypted in the local store (nothing lost, nothing leaked)
END
```

Success condition. $|Q| = 0 \wedge |W| = 0 \wedge$ every record id $\in L \wedge$ every P0 event notified $\leq$ deadline.

Failure conditions. (a) a plaintext record is observed outside the secure store; (b) a P0 event is dropped without a notification; (c) work for a logged-out user is still running; (d) a record is acknowledged twice.

Why this is more than "use WorkManager". The pseudocode makes five decisions explicit that a one-line answer cannot: *which* item next (SELECT_NEXT), *which* component (CHOOSE_COMPONENT), *whether* it is allowed (FEASIBLE), *how* to leave a clean state (UNDO), and *when* the whole system is correct (success / failure condition).

Architecture diagram (component and Intent view).

[DIAGRAM A1 – inserted at the end of this task]

A1.5 – Trace the Design

Question. Construct a scenario with at least five distinct events (sensor reading, connectivity change, Doze entry, critical alert, user logout) and manually trace Choose → Check Constraint → Execute / Defer → Notify → Cancel / Backtrack.

Scenario set-up. MAX_WORKERS = 3, battery 62 %, Wi-Fi good, not Doze. Records: R1 (P2, 3 KB), R2 (P2, 4 KB), R3 (P0 critical, 1 KB), R4 (P3, 8 KB).

t	Event	Choose (SELECT_NEXT)	Check constraint	Execute / Defer	Notify	Cancel / Backtrack	State σ after the step
0	**Sensor reading** R1 (P2) arrives, encrypted	R1	PERM✓ ENC✓ NET good✓ BAT 62 %≥20✓ Doze✗ not active✓	**Execute**: UploadWorker started with R1	–	–	Q={}, W={R1}
1	Sensor reading R2 (P2) arrives	R2	all ✓	**Execute** R2 (second worker)	–	–	Q={}, W={R1,R2}
2	**Connectivity change**: network → none	(no new item); running workers stall	NET(R1, R2)=✗	R1 and R2 are **paused with checkpoint** C[R1]=chunk 1 of 2, C[R2]=chunk 0	–	WorkManager retry scheduled with back-off	Q={R1,R2 (notBefore =t+2)}, W={}
3	Sensor reading R4 (P3, 8 KB) arrives	R4	NET ✗	**Defer** R4 (reason NET)	–	–	Q={R1,R2 ,R4} all deferred
4	**Doze entry**	(candidates: R1,R2,R4)	DOZE ✗ for P1–P3	**Defer** all (no attempt, no wake-up)	–	–	Q unchanged; wake-ups avoided = 3

t	Event	Choose (SELECT_NEXT)	Check constraint	Execute / Defer	Notify	Cancel / Backtrack	State σ after the step
5	**Critical alert**: $SpO_2 = 85$ %, R3 (P0)	R3 (score 30 + … highest)	PERM✓ ENC✓ NET ✗ (none) BAT✓ DOZE exempt (expedited)	R3 cannot upload (no network) → **Defer upload** but **Notify now**	**Local heads-up notification + ordered broadcast to wearable bridge**	–	Q={R1,R2,R3,R4}; alert handled locally (goal condition 3 met)
9	**Connectivity restored** (good), Doze still on	SELECT_NEXT → R3 first	R3: DOZE exempt ✓ NET ✓	**Execute** R3 as *expedited* work	"Alert delivered to clinician" status	–	W={R3}; R1,R2,R4 stay deferred (DOZE)
10	Doze exit (maintenance window)	R1 (age high), R2, R4	all ✓ (R4: size 8 > 6 needs good net ✓)	**Execute** R1, R2 (cap 3 → R4 waits one pass)	–	–	W={R1,R2,R3}, Q={R4}
12	**User logout**	–	PERM(E) =✗ (session invalid)	–	–	**Cancel** W={R1, R2,R3}: UNDO aborts upload sessions, marks records pending, wipes keys;	σ = ∅ for this user; encrypted records kept per policy

t	Event	Choose (SELECT_NEXT)	Check constraint	Execute / Defer	Notify	Cancel / Backtrack	State σ after the step
						purge Q={R4}	

What the trace demonstrates.

1. *Choose* is priority-aware: the critical R3 jumps the queue at t = 5 and t = 9.
2. *Check constraint* produces three different outcomes – defer (NET, DOZE), exempt (P0 under Doze), cancel (PERM).
3. *Execute / Defer* – at t = 4 three wake-ups were avoided by deferring rather than trying and failing.
4. *Notify* is decoupled from upload: the user is alerted at t = 5 even though the network is down.
5. *Cancel / Backtrack* – at t = 12 UNDO restores a clean state: no running worker, no plaintext, no lost record.

Evidence provided for Task A1: problem model (A1.1), state definition (A1.2), constraint conditions (A1.3), complete pseudocode (A1.4), execution trace (A1.5), component diagram.

TASK A2 – Don't Synchronise What Cannot Be Safe

Question. The naive synchronisation algorithm below uploads every record and simply continues when the network fails. Diagnose it, redesign it incrementally with constraint pruning, demonstrate the pruning on five records and challenge the pruning rule under offline-first / eventual-consistency requirements.

```
ALGORITHM SyncAll(LocalStore, RemoteAPI)
  FOR each record R in LocalStore DO
    Upload R to RemoteAPI
    IF network fails THEN CONTINUE
```

END FOR

END

A2.1 – Diagnose the Computational & Security Waste

Answer. The algorithm postpones every decision until after the work has been attempted. Each defect below is a place where energy, bandwidth or privacy is lost.

#	Defect in SyncAll	Type of waste	Consequence
1	No *dirty flag* / delta: every record is considered on every run, including records already synchronised	Computation + bandwidth	Work grows with the **size of the store**, not with the amount of **new** data
2	Attempt-first: the upload is started *before* checking network class	Energy	Each failed attempt wakes the radio and keeps it in a high-power state (the "tail") for several seconds; a failed attempt costs almost as much energy as a successful one
3	CONTINUE after failure: no retry, no back-off, no record of what failed	Correctness	Failed records are silently skipped; the next run may fail the same way; data may never reach the server
4	No priority: a 3 MB image is sent before a critical vital-sign reading	Usability / safety	Critical data waits behind bulk data on a weak link
5	No battery or charging test	Energy	Sync continues at 5 % battery and can drain the device to shutdown
6	No metered-network / Wi-Fi-only rule	Money / privacy of user preferences	Large uploads consume cellular data the user did not authorise

#	Defect in SyncAll	Type of waste	Consequence
7	No encryption check	**Security leak**	A record stored in plaintext can be uploaded over an unsafe channel
8	No conflict detection (no version / ETag)	Data integrity	A stale local copy overwrites a newer server copy (lost update)
9	No checkpoint / idempotency key	Duplicate work	After a mid-transfer failure the whole record is re-sent, and a lost ACK produces a duplicate on the server
10	No batching, one request per record	Energy / latency	Each request pays connection set-up and TLS handshake cost

Quantified example. With 5 pending records and a link that fails for 3 of them, SyncAll performs 5 radio wake-ups and loses 3 records from the run; a feasibility-first design performs 1 wake-up (a single batch of the feasible record) and keeps 4 records safely queued.

A2.2 – Redesign Using Incremental & Constraint-Aware Synchronisation

Answer. The redesigned algorithm builds the **synchronisation set S** one record at a time. For each record it explicitly represents the alternatives **Upload → Defer → Discard / Resolve-conflict**, and it applies the cheapest tests first (a test that costs nothing is evaluated before a test that costs a network round-trip).

```
ALGORITHM ConstraintAwareSync(LocalStore, RemoteAPI, E)
// E = (battery, charging, network_class, metered, keystore_unlocked, session_valid)
// S = synchronisation set built incrementally;  D = deferred set;  X = discarded set

CONSTANT  BATCH_BYTES = 256 KB,  THETA = { P0:0, P1:20, P2:20, P3:30 }   // battery %

S <- EMPTY ;  D <- EMPTY ;  X <- EMPTY ;  bytes <- 0
R_list <- SORT( { r IN LocalStore WHERE r.dirty = TRUE },
```

```
            BY priority DESC, THEN age DESC, THEN size ASC )   // delta + order

IF NOT E.session_valid THEN RETURN (S, D, X)                   // nothing may leave

FOR EACH r IN R_list DO

   // ---------- level 0: structural / free tests ----------
   IF r.sensitivity = HIGH AND NOT r.encrypted THEN
        IF E.keystore_unlocked THEN r <- ENCRYPT(r)          // repair, then continue
        ELSE  D <- D + {r} ; LOG(r,'defer','ENC') ; CONTINUE  // cannot send plaintext
   IF r.expired AND r.priority <> P0 THEN  X <- X + {r} ; CONTINUE // DISCARD stale

   // ---------- level 1: environment pruning ----------
   IF E.network_class = NONE THEN
        D <- D + {r} + REST(R_list after r)      // PRUNE the whole remaining branch
        BREAK
   IF E.battery < THETA[r.priority] AND NOT E.charging THEN
        // thresholds grow as priority falls: every later record also fails
        D <- D + {r} + { q IN REST WHERE q.priority <= r.priority }
        BREAK
   IF E.metered AND r.size > SMALL AND r.priority > P1 THEN
        D <- D + {r} ; CONTINUE                  // wait for Wi-Fi
   IF E.network_class = POOR AND r.size > 6 KB AND r.priority <> P0 THEN
        D <- D + {r} ; CONTINUE

   // ---------- level 2: conflict check (costs one cheap HEAD request) ----------
   v <- REMOTE_VERSION(r.id)
   IF v > r.base_version THEN
        r <- RESOLVE(r, v)                       // merge by field rules / ask user
        IF r = UNRESOLVABLE THEN  D <- D + {r} ; CONTINUE
   IF r.id IN L (ledger) THEN  X <- X + {r} ; CONTINUE     // already delivered
```

```
    // ---------- level 3: admit to the set ----------
    IF bytes + r.size > BATCH_BYTES THEN  EXECUTE(S) ; S <- EMPTY ; bytes <- 0
    S <- S + {r} ; bytes <- bytes + r.size          // UPLOAD alternative chosen
END FOR

EXECUTE(S)    // one encrypted, resumable batch request; ACK -> add ids to ledger L, clear dirty
RETURN (S, D, X)
END ALGORITHM
```

Where each constraint prunes.

Constraint	Test	Pruning effect
Network quality	network_class = NONE	Whole remaining branch deferred in one step (no per-record attempt)
Battery level	battery < θ(priority)	Because θ is *monotone* in priority, the first failure proves that all lower-priority records also fail → terminate the loop
Record priority	Sort order + critical exemption	Critical records are examined first and are never pruned by battery or Doze
Encryption status	sensitivity = HIGH ∧ ¬encrypted	Plaintext is never admitted to S; encrypt-or-defer

---

A2.3 – Demonstrate Pruning (1 Mark)

Answer. Concrete data set (priority ↑ = more urgent):

Record	Content	Priority	Size	Sensitivity	Encrypted?	Dirty?
R1	Vital-sign alert (SpO_2 85 %)	P0	2 KB	High	Yes	Yes
R2	Daily summary	P1	40 KB	Medium	Yes	Yes
R3	Patient note	P1	5 KB	High	**No** (Keystore locked)	Yes
R4	Raw accelerometer log	P3	900 KB	Low	Yes	Yes
R5	Diagnostic image	P3	3 MB	High	Yes	Yes

Environment: battery **18 %**, not charging, network **POOR** (about 0.3 Mbps), metered.

Sorted by priority then age: R1, R2, R3, R4, R5.

Step	Record	Tests	Decision	Search nodes visited
1	R1 (P0)	ENC ✓; battery test exempt ($\theta(P0) = 0$); poor-network rule exempt for P0; conflict none	**UPLOAD** → S = {R1}	1
2	R2 (P1)	ENC ✓; battery 18 % < $\theta(P1) = 20$ % → **fails**	**DEFER and PRUNE the branch**: D = {R2, R3, R4, R5}; loop terminates	1

The early-terminated branch. After step 2 the algorithm never examines R3, R4 and R5. This is safe because:

1. $\theta(P3) = 30\ \% \geq \theta(P1) = 20\ \%$, so R4 and R5 would also fail the battery test.
2. R3 additionally fails ENC (plaintext, Keystore locked) and R5 additionally fails the poor-network/metered rule (3 MB).

3. Each of these three has *at least one* independent reason to be infeasible; a subset of S that included any of them would violate a constraint, so **no correct or safe synchronisation set exists in that sub-tree**.

Search-space reduction. If each of the 5 records may be Uploaded, Deferred or Discarded, the full decision tree has $(3^6 - 1) / 2 =$ **364 nodes**. With pruning, the algorithm visits **2 decision nodes** and obtains the unique maximal safe set S = {R1}. The result: 1 batch, 2 KB sent, 0 plaintext bytes sent, 4 records safely retained.

A2.4 – Challenge Your Pruning Rule

Question. Suppose offline-first operation with eventual consistency is now required. Does the pruning condition remain valid? If not, which assumption has been violated?

Answer. Part of it remains valid and part does not.

Still valid (safety rules). The ENC rule and the PERM rule: plaintext must never leave the device and no work runs without permission, regardless of the consistency model. The battery and network tests also remain valid as conditions on *when to transmit*.

No longer valid (liveness / discard rules), and the violated assumptions:

Violated assumption	Why it held in A2.2	Why it fails offline-first	Required change
A1 – "The environment is static during the pass." Pruning a branch is *final* because we assumed E would not improve within this pass	A sync pass is short	Offline-first systems run for hours without network; E *will* improve (Wi-Fi + charging)	Pruning must be **reversible deferral with re-entry**: the branch is re-opened on every NET/BAT change; BREAK becomes "suspend until state change"

Violated assumption	Why it held in A2.2	Why it fails offline-first	Required change
A2 – "The server is the authority." RESOLVE or DISCARD could use the server copy	Single-master model	Replicas diverge legitimately; discarding a local update loses a user's offline edit	Replace discard with **merge** (version vectors / CRDT / field-level last-writer-wins with tombstones); **never discard a local write**
A3 – "Stale means worthless." r.expired → DISCARD	Telemetry loses value with age	In eventual consistency a write must *eventually* be applied, however old	Remove TTL-based discard for user data; keep TTL only for ephemeral notifications
A4 – "Conflict detection needs one server round-trip per record."	Online sync	Offline, versions can only be compared at merge time	Store a base version with each local write; compare during the merge step, not at the pruning step

Precise statement. The pruning condition *"prune the remaining branch and terminate"* is valid only under the assumption that **the feasibility of a record cannot change before the next full pass**. Under offline-first operation that assumption is false, so terminating the branch permanently would violate eventual consistency (some records could stay unsynchronised forever). The corrected rule is: **prune = defer-with-re-entry, never discard**.

Evidence provided for Task A2: waste analysis (A2.1), redesigned pseudocode (A2.2), pruning trace with node count (A2.3), reasoning under changed requirements (A2.4).

TASK A3 – Discover the Mobile-Background Design Pattern

Question. Three apparently different problems are given: *Background Sensor Collection* → *Offline Data Synchronisation* → *Push Notification Delivery.* Complete the design-analysis table,

identify the common pattern, develop a generic SCHEDULE_OR_EXECUTE(State) framework and analyse the role of undo / cancellation.

Completed Design-Analysis Table

Design Component	Sensor Collection	Data Sync	Push Notification
State	Sensor handle, sampling rate, in-memory buffer, battery %, permission flag, foreground-service status	Dirty-record set, server version per record, ledger of acknowledged ids, network class, upload checkpoint	Pending alerts, delivered-id set, notification-channel settings, push token validity, Do-Not-Disturb state
Decision variable	Sampling interval and batch size (and whether to keep the sensor registered)	Which record to put in the next upload batch and when to send it	Notify now, batch into a digest, or suppress; which channel / importance
Candidate choices	{sample at high rate, sample at low rate, batch-sample, stop sensor}	{upload, defer, discard stale, merge on conflict}	{heads-up alert, silent update, daily digest, escalate through ordered broadcast, drop duplicate}
Feasibility / Constraint test	Permission granted; sensor available; battery $\geq \theta$; foreground-service type declared (Android 12+ start rules)	PERM ∧ ENC ∧ NET ∧ BAT ∧ Doze-exempt-or-inactive ∧ version check	POST_NOTIFICATIONS granted; channel not blocked; DND policy allows the priority (critical may bypass); token valid; not a duplicate; rate-limit not exceeded
Goal condition	Every required reading captured at the required	Every dirty record acknowledged	Every critical event seen by the user within its deadline,

Design Component	Sensor Collection	Data Sync	Push Notification
	rate within the energy budget	exactly once (or safely retained encrypted)	no duplicates, no leak on lock screen
Pruning / Deferral condition	Battery below θ → lower the rate or unregister non-essential sensors; screen-off + no motion → batch	No network, Doze, low battery, metered + large payload → defer with back-off	Quiet hours or DND for non-critical → defer to digest; same event within the coalescing window → merge
Undo / Cancel operation	Unregister the listener, release the wake lock, flush the buffer to the encrypted store, stop the foreground service	Abort the upload session, set *in-flight* records back to *pending*, delete temporary buffers	Cancel the notification id, cancel the scheduled alarm / pending intent, clear the stale content on logout

A3.1 – Identify the Common Pattern

Answer. Although the three problems use different Android APIs (`SensorManager`, `WorkManager` + REST, FCM + `NotificationManager`), the table shows that each one has the **same seven columns filled with the same kind of content**. Computationally, each is a **constrained sequential decision problem over a queue of work items**, and architecturally each follows the *choose* → *test* → *commit* → *undo* shape of **backtracking search with pruning**:

1. **A state that evolves over time** – resources (battery, network, permission) plus work-in-progress.
2. **A choice at each step** among a small set of candidates, one of which is always "do nothing now" (*defer*).
3. **A feasibility test that is cheap compared with the action** – so testing before acting saves energy.

4. **Monotone pruning rules** – when a high-priority item fails a resource test, every lower-priority item fails it too, so a whole branch is cut.
5. **Reversible commits** – every action has an inverse (undo / cancel), because the environment may change under the running work.
6. **A goal condition that is a set of invariants**, not a single value (safety: nothing leaks; liveness: critical work is delivered).
7. **Event-driven re-evaluation** – the search is re-entered whenever the environment changes (network up, Doze exit, charger connected).

Put differently, the three problems are **three instances of one scheduler** whose *candidate set* and *constraint set* differ but whose *control structure* is identical. That is why a generic framework can serve all three.

Mapping to a classical idea. The pattern is a *constraint-satisfaction* problem solved by *depth-first search with forward checking* where the "depth" is the sequence of time-ordered decisions and *defer* is the backtrack-and-retry-later move.

A3.2 – Design a Generic Background-Work Framework

Answer. The framework is parameterised by five functions supplied by each problem: CANDIDATES, FEASIBLE, SCORE, APPLY and UNDO.

```
FRAMEWORK BackgroundWork
  PARAMETERS (supplied per problem):
     CANDIDATES(State) -> list of choices
     FEASIBLE(State, c) -> (TRUE | FALSE, reason, kind)   // kind = TRANSIENT |
PERMANENT
     SCORE(State, c)    -> number                  // selection criterion
     APPLY(State, c)    -> State'                  // start / enqueue / send
     UNDO(State, c)     -> State''                 // cancel / restore
     GOAL(State)        -> boolean
     ENV_CHANGED(State) -> boolean
```

```
PROCEDURE SCHEDULE_OR_EXECUTE(State)
  IF GOAL(State) THEN RETURN SUCCESS

  LOOP
    C <- CANDIDATES(State)
    IF C = EMPTY THEN  RETURN WAIT_FOR(ENV_CHANGED)       // nothing to do:
sleep, no polling

    SORT C BY SCORE DESCENDING
    progressed <- FALSE

    FOR EACH c IN C DO
      (ok, reason, kind) <- FEASIBLE(State, c)

      IF ok THEN                                  // EXECUTE
        State <- APPLY(State, c)
        REGISTER callback ON_RESULT(c)
        progressed <- TRUE

      ELSE IF kind = TRANSIENT THEN                    // DEFER (prune, keep item)
        SCHEDULE c WITH notBefore = NEXT_STATE_CHANGE_OR_BACKOFF
        IF reason IS MONOTONE_IN_SCORE THEN          // e.g. battery threshold
          DEFER all items with SCORE <= SCORE(c) ; BREAK

      ELSE                                        // PERMANENT: cancel / reject
        State <- UNDO(State, c)
        MARK c rejected(reason)
    END FOR

    IF NOT progressed THEN RETURN WAIT_FOR(ENV_CHANGED)
  END LOOP
```

```
END PROCEDURE

PROCEDURE ON_RESULT(State, c, result)
  IF result = SUCCESS         THEN State <- COMMIT(State, c)
  ELSE IF result = RETRYABLE  THEN State <- UNDO(State, c) ; RE-QUEUE c with back-off
  ELSE                        State <- UNDO(State, c) ; FAIL(c)
  CALL SCHEDULE_OR_EXECUTE(State)
END PROCEDURE
```

How the same framework represents the three problems.

Framework hook	Sensor Collection	Data Sync	Push Notification
CANDIDATES	rate settings and sensors to keep registered	dirty records	undelivered alerts
FEASIBLE	permission ∧ battery ≥ θ	PERM ∧ ENC ∧ NET ∧ BAT	channel enabled ∧ DND allows
SCORE	clinical importance of the sensor	priority + age/TTL + short-job bonus	severity
APPLY	register listener / start foreground service	start upload batch	post notification
UNDO	unregister, release wake lock, flush buffer	abort session, in-flight → pending	cancel notification id
GOAL	required rate achieved within budget	dirty set empty	all critical alerts delivered

---

A3.3 – Analyse the Role of Undo / Cancellation

Answer. A background task changes the world in *several* places before it finishes: it takes a wake lock, opens a socket, marks records as in-flight, creates a server-side upload session and may write

temporary files. If the task is cancelled or fails (Doze entry, permission revoked, network lost, logout), the system is left in a **half-committed state**. Without UNDO the half-committed state becomes the permanent state, which breaks both safety and liveness. Undo is therefore not a convenience; it is what makes retry and cancellation *safe*.

Illustration with Data Sync (what happens if UNDO is omitted).

Omitted step	Resulting fault	Violated property
Reset *in-flight* → *pending*	The record stays marked in-flight forever; the scheduler never selects it again	**Liveness / data loss** – a clinical record is silently never uploaded
Abort the server upload session	The server holds a partial object; a later retry creates a second one	**Correctness** – duplicate or corrupt record
Delete temporary plaintext buffer	Decrypted copy remains in cache after logout	**Security** – privacy leak
Release wake lock / close socket	CPU and radio stay awake	**Efficiency** – battery drain

Conclusion. A framework whose only operation is *start* can finish, but cannot be *correct under cancellation*. Because mobile environments change continuously, every APPLY in the framework must have a matching UNDO, and every cancellation path must call it.

Evidence provided for Task A3: completed comparison table, generic pseudocode SCHEDULE_OR_EXECUTE, conceptual explanation of the shared pattern and the role of undo.

TASK A4 – Tractability: Local vs Cloud vs Hybrid Processing

Question. For each of six mobile-computing problems prepare a concise analysis: what constitutes a solution, whether a proposed design can be efficiently verified, the complexity / resource class and the reason. Then answer the two inquiry questions.

Resource classes: On-device feasible | Network-bound | Battery-critical | Security-critical | Hybrid required.

How to read "verification" here. For a mobile design, *finding* a good design may be hard, but *checking* a proposed design is usually cheap: run it on a test workload and compare with a target. The interesting cases are those where checking is **not** cheap or not conclusive (for example, a security property cannot be established by running tests).

Analysis Table

#	Problem	What constitutes a solution?	Can a proposed design be efficiently verified?	Complexity / Resource class	Reason
1	On-device image classification (tiny model)	A predicted label whose top-1 accuracy meets the target (e.g. ≥ 90 % on a held-out set) with latency ≤ 50 ms per image	**Yes.** Run the model on a test set; accuracy and latency are measured directly in time linear in the test-set size	**On-device feasible** – inference cost O(number of multiply-accumulates), a few MFLOPs for a quantised mobile network	The model fits in memory and runs on the NPU / GPU / CPU; no network needed, no data leaves the device, energy per inference is small
2	Cloud-based speech recognition	A transcript whose word-error rate (WER) is below the target, delivered within the latency budget (e.g. first partial result ≤ 300 ms)	**Yes for accuracy** (compare with a reference transcript); **latency is verified only under the real network distribution**, so it must be measured, not computed	**Network-bound** – cost = audio upload + round-trip time + server inference	The acoustic / language models are hundreds of MB and need server accelerators; the device only streams audio, so end-to-end time is dominated by the network and not by the phone's CPU

#	Problem	What constitutes a solution?	Can a proposed design be efficiently verified?	Complexity / Resource class	Reason
3	Peer-to-peer file sharing among nearby devices	The complete file reaches the peer with a matching checksum, without infrastructure	**Yes.** A hash comparison verifies integrity in O(file size); discovery success can be tested in the lab	**Battery-critical** – radio scanning (BLE / Wi-Fi Direct discovery) and sustained transfer dominate energy	No server is needed, so the cost shifts to both phones' radios; discovery scanning and long transfers can drain both batteries, so duty-cycling and charge-aware scheduling decide feasibility
4	End-to-end encrypted messaging	Only the two endpoints can read the message; messages arrive in order, are authenticated and keys are forward-secret	**Partly.** Functional correctness (decrypt-and-verify tag) is cheap to check; the *security property* (no third party can read it) **cannot be verified by testing** – it needs protocol analysis and audit	**Security-critical** – computation is cheap (ECDH + AES-GCM in microseconds); the hard part is key management and the trust model	A correct-looking build can still leak (weak randomness, key backup in the cloud, metadata). The cost of error is high and not observable by performance measurement
5	Real-time multiplayer game state synchronisation	All clients converge to a consistent state, with perceived input delay ≤ 100 ms and no cheating	**Consistency: yes** (compare state hashes). **Latency and fairness: only statistically** (depends on the	**Hybrid required** – authoritative server for truth and anti-cheat + client-side prediction / interpolation	Pure cloud makes every action wait for a round-trip; pure local cannot be trusted or synchronised. Splitting the work gives both low

#	Problem	What constitutes a solution?	Can a proposed design be efficiently verified?	Complexity / Resource class	Reason
			live network distribution)	for responsiveness	latency and authority
6	Large-scale offline map rendering with vector tiles	A 60 fps (≤ 16 ms/frame) render of the viewport from locally stored tiles, within a memory cap	**Yes.** Frame-time, GPU memory and tile-cache hit-rate are directly measurable on a benchmark pan / zoom path	**On-device feasible** (GPU- and memory-bound; the heavy step is *tile decode and tessellation*)	Offline operation removes the network entirely; vector tiles are compact, and the problem becomes cache sizing, level-of-detail and GPU batching, which a phone can do

Summary by class:

Resource class	Problems
On-device feasible	1, 6
Network-bound	2
Battery-critical	3
Security-critical	4
Hybrid required	5

Inquiry 1 – Why is "runs slowly on a phone" not an adequate definition of a performance problem?

Answer. The sentence names neither a **resource**, a **metric**, a **workload**, a **device** nor a **target**, so it cannot be measured, reproduced or fixed. A mobile performance problem must be stated as

metric (percentile) on a device class, under a workload and condition, versus a target.

Five reasons why the vague statement fails:

1. **"Slow" is perceptual, not absolute.** A task is slow only relative to a human threshold: 16 ms per frame for smooth scrolling, about 100 ms for a tap response, 5 s for an ANR (Application Not Responding) dialog.
2. **Which resource?** The same symptom can come from CPU, GPU, memory pressure (garbage collection), disk I/O, the network, thermal throttling or a lock on the main thread. Each has a different remedy (move work to a background thread, reduce overdraw, cache, compress, batch).
3. **Which device?** The same code is fast on a flagship and slow on a low-end phone; "the phone" is not one device. A fix measured on the wrong device class proves nothing.
4. **Which condition?** Cold vs warm start, low battery (CPU governor), thermal throttle after 10 minutes, 3G vs Wi-Fi: the average hides the p95 or p99 that users actually notice.
5. **Slowness trades against other goals.** A change that makes the screen faster may double energy use; "slow" does not tell the engineer which trade-off is acceptable.

Example of a well-formed statement. "On a mid-range device (4 GB RAM), scrolling the image feed for 60 s on 4G causes more than 5 % of frames to exceed 16 ms and raises battery drain above 3 % per 10 minutes." This names the metric (frame time, drain), the percentile, the workload, the device and the target, and therefore can be investigated with Perfetto or the Android Studio Profiler.

Inquiry 2 – Why is the distinction between local processing and cloud offloading important for scalability and security?

Answer. The two placements shift **where the cost and where the trust live**, and those shifts change how the system behaves as users grow.

Aspect	Local processing	Cloud offloading
Scalability (compute)	Each new user brings their own CPU, so total compute scales with users at **no server**	Server load grows with users; needs autoscaling, queueing and cost

Aspect	Local processing	Cloud offloading
	cost; limited by the *weakest* device	control; **a popular feature can overload the backend**
Scalability (devices)	Must run across hundreds of hardware profiles (device fragmentation)	One execution environment, easy to update and optimise
Latency	No network round-trip; works offline	Adds RTT + transfer time; unusable without a connection
Energy	Spends CPU energy	Spends radio energy; wins only when the saved computation exceeds the transmission cost (formula below)
Security / privacy	Data stays on the device: small attack surface, data minimisation; risks are device theft, malware, rooted devices	Data crosses the network and rests on servers: needs TLS, access control, retention rules; **one breach exposes all users**
Update / patching	Needs an app release and user adoption	Fixed centrally, instantly

Break-even rule for offloading (energy view). Let C be the computation (instructions), D the data to be sent, S_m and S_s the mobile and server speeds, B the link bandwidth, P_c the compute power, P_t the transmit power and P_i the idle (waiting) power:

$$E_{local} = P_c \cdot \frac{C}{S_m} \qquad E_{offload} = P_t \cdot \frac{D}{B} + P_i \cdot \frac{C}{S_s}$$

$$\text{Offload only if } E_{local} > E_{offload}$$

If the data D is large and the computation C is small, local processing wins; if C is large and D is small (speech recognition), offloading wins. Since B varies strongly with connectivity, the right

answer changes during the day, which is exactly why mobile applications need **hybrid** designs and why “it works on the server” does not describe the performance the user will see.

Without this distinction an engineer could interpret a load test on the server as evidence of scalability, miss the user-side battery cost, or send sensitive health data to the cloud when a local model would meet the requirement.

Evidence provided for Task A4: correct classification with reasoning about verification, resource constraints and problem formulation.

TASK A5 – Design When Exact Optimisation May Be Too Expensive

Question. A social-media application must render a continuously scrolling feed of images and short videos on devices from low-end to flagship. The objective is to keep frame time under 16 ms while minimising battery drain and memory pressure.

A5.1 – Exact Perspective

Question. Describe how an exact combinatorial search over decoding resolution, cache size and prefetch distance could identify an optimal configuration. Analyse why it becomes problematic as the number of feed items and device profiles increases.

Answer.

Formulation. For a visible window of n feed items choose

- a decoding resolution $r_i \in R$ for every item i (for example R = {240p, 480p, 720p, 1080p}, |R| = 4),
- one cache size $c \in C$ (for example 20, 40, 60 MB),
- one prefetch distance $d \in D$ (for example 0, 1, 2 screens ahead).

Minimise the cost

J(r, c, d) = \sum_{i=1}^{n} \Big[\alpha \cdot \text{drop}_i(r_i, c, d) + \beta \cdot \text{energy}_i(r_i, d) + \gamma \cdot (r_{max} - r_i) \Big]

\text{subject to}\quad \text{frame_time}_i \le 16\ \text{ms}, \qquad \sum_i \text{mem}_i(r_i) + \text{mem}_{cache}(c) \le M_{device}

Exact procedure. Enumerate every triple (r, c, d), simulate the scroll trace for that configuration, record frame drops, energy and memory, discard configurations that violate the memory limit, and keep the one with minimum *J*. This is guaranteed to find the optimum for the given trace and cost weights.

Why it breaks down.

Growth driver	Effect on the search	Example
Number of feed items *n*	The number of configurations is **	R
Device profiles *K*	The search must be repeated per profile (CPU, GPU, RAM, screen refresh rate), multiplying the cost by *K* (hundreds of Android models)	300 profiles × any of the above
Memory constraint	The problem becomes a *multiple-choice knapsack* (pick one resolution per item under a memory budget), which is NP-hard in general	Dynamic programming is only pseudo-polynomial in the memory limit
Unknown future workload	The optimum is defined for a *known* scroll trace, but the user's scrolling, the content mix and the thermal state change continuously	The "optimal" configuration is out of date before it has been computed
Cost of one evaluation	Each candidate needs a simulation or a real measurement of many frames	Even 1 ms per evaluation makes 10^{13} candidates take about 300 years

Conclusion. The exact search is a useful *offline benchmark* (and Part B3 uses it on small instances for that purpose), but it cannot be run on the device, per item, in real time.

A5.2 – Approximation / Heuristic Design

Question. Design pseudocode for a greedy / adaptive strategy that repeatedly adjusts decoding quality, cache eviction and prefetch window based on measured frame time and battery level. It must maintain current metrics, selected parameters, selection criterion, state update and termination / steady-state condition.

Answer. The strategy is a **feedback controller**: measure, compare with the target, make the single *cheapest local* change that most improves the violated metric, and repeat until a steady state is reached.

```
ALGORITHM AdaptiveFeedTuner

STATE (maintained):
  metrics   M = ( frame_ms_p95, drop_rate, battery_pct, mem_used, cache_hit_rate )
  params    P = ( quality[i] for each visible item i,  cache_size c,  prefetch_window d )
  history   H = last 3 parameter vectors (to prevent oscillation)

CONSTANT  FRAME_TARGET = 16 ms,  SLACK = 0.7 * FRAME_TARGET,
LOW_BATTERY = 30 %
      QUALITY_LEVELS = [1..4],  CACHE_LEVELS = [1..3],  PREFETCH_LEVELS =
[0..2]

INITIALISE P <- ( all items at highest affordable quality for device class, c = min, d = min )

REPEAT every measurement window (e.g. every 30 frames)

  M <- MEASURE()                          // current performance metrics

  // ---------- 1. SELECTION CRITERION: what is the worst violated goal? ----------
  IF M.frame_ms_p95 > FRAME_TARGET THEN          // jank is the first priority
      i <- ITEM with the largest estimated frame cost
```

```
        IF quality[i] > min THEN  quality[i] <- quality[i] - 1       // cheapest fix
        ELSE IF c < max AND M.mem_used < MEM_LIMIT THEN  c <- c + 1  // raise hit rate
        ELSE IF d > min THEN d <- d - 1                              // reduce decode pressure

    ELSE IF M.mem_used > MEM_LIMIT THEN                  // memory pressure
        EVICT least-recently-used bitmaps until mem_used <= 0.9 * MEM_LIMIT
        j <- ITEM with the largest memory use
        IF quality[j] > min THEN quality[j] <- quality[j] - 1

    ELSE IF M.battery_pct < LOW_BATTERY THEN             // energy saving mode
        IF d > min THEN d <- d - 1                   // prefetch costs radio and CPU
        ELSE IF some item has quality > 2 THEN lower the highest-quality item by 1

    ELSE IF M.frame_ms_p95 < SLACK AND M.mem_used < 0.8 * MEM_LIMIT THEN
        k <- ITEM with the lowest quality
        IF quality[k] < max THEN  TRY quality[k] + 1    // spend slack on image quality
            REJECT the try if the predicted frame_ms > FRAME_TARGET

    // ---------- 2. STATE UPDATE ----------
    IF new P is in H THEN  FREEZE the last change (anti-oscillation)
    ELSE  APPLY P ; ADD P TO H

    // ---------- 3. TERMINATION / STEADY STATE ----------
UNTIL  M.frame_ms_p95 <= FRAME_TARGET AND M.mem_used <= MEM_LIMIT
       AND no parameter changed in the last 3 windows          // steady state
```

RE-ENTER on any of: scroll speed change, new content type, thermal throttle, battery drop below 30 %

Why the strategy is cheap. Each iteration changes *one* parameter and evaluates *one* measurement, so the work per window is O(n) (to find the worst item) and the number of windows to settle is bounded by the number of available levels, $O(n \cdot |R|)$, not $|R|^n$.

A5.3 – What Have We Sacrificed?

Question. Analyse the distinction between *finding a feasible, smooth scrolling experience* and *proving that the chosen parameters minimise total energy for a given quality of service.*

Answer.

	Feasible, smooth scrolling	Provably minimum energy for a given QoS
Question answered	"Is frame time ≤ 16 ms and memory within the limit?"	"Is there *any* configuration that gives the same QoS with less energy?"
Logical form	**Existence** – one witness suffices; the controller *is* the witness	**Universal** – must rule out *all* other configurations
Cost to establish	One measurement window per step	Exhaustive or dual-bound proof over an exponential space
Verifiable on the device	Yes, by measuring frame time	No; would need the cost model of every alternative
What the greedy controller guarantees	Reaches a *local* steady state that satisfies the constraints (if one is reachable by one-step moves)	Nothing: a one-step-at-a-time rule can stop at a local optimum

What is sacrificed. The greedy controller gives up the **optimality guarantee** and the **independence from the starting point**. It may keep a high decoding quality on one item because lowering it was not the *first* move its rule examined, while a different combination (lower quality on a different item and a bigger cache) would use less energy at the same smoothness. Part B3 constructs an explicit counter-example and measures how large that gap can be.

What is gained. Predictable, bounded work per frame window, no need for a model of every device, and the ability to react to thermal and battery changes that no offline search could foresee.

For a feed, users notice dropped frames immediately but cannot perceive a few percent of extra energy, so "smooth and good enough" is the right engineering target; a proof of minimum energy would be paid for by every user and noticed by none.

Evidence provided for Task A5: exact-search reasoning, approximation pseudocode, explanation of the optimality trade-off.

PART B – IMPLEMENTATION & COMPUTATIONAL INVESTIGATION

Part B asks whether the architectural reasoning of Part A survives contact with actual computation. The workflow is **Implement → Instrument → Stress → Measure → Compare → Interpret**.

Experimental set-up (common to B1–B4).

Item	Value
Android components	Kotlin classes (`WorkItem`, `ConstraintChecker`, `SchedulerPolicy`, `UploadWorker`, `Metrics`) shown in B1
Measurement harness	Python 3 discrete-time simulator that implements exactly the same FEASIBLE, SELECT_NEXT, UNDO logic as the Kotlin classes (one tick = one scheduler wake-up)
Environment model	Battery, network class (good / poor / none), Doze flag and permission flag as functions of the tick; three named scenarios S1, S2, S3 (below)
Concurrency cap	MAX_WORKERS = 3
Bandwidth model	good = 4 KB/tick, poor = 1 KB/tick, none = 0
Time-to-live	P0 = 12 ticks, P1 = 25, P2 = 40, P3 = 60
Randomness	Fixed seeds, stated beside every experiment
Limit of the evidence	The numbers are simulator outputs for the stated models, **not** readings from a physical phone

TASK B1 – Implement and Instrument Your Background Architecture

Question. Implement the constraint-aware background design of Task A1. The program must record: work items enqueued; items deferred / cancelled by constraints; successful uploads; battery and network state at each decision point; maximum concurrent workers; and the final outcome (all data delivered or residual backlog).

Source Code (Kotlin – Android components)

```
// ---------- Model ----------
enum class Priority(val rank: Int, val ttlTicks: Int) { P0(3, 12), P1(2, 25), P2(1, 40), P3(0, 60) }

data class WorkItem(
   val id: Long, val priority: Priority, val sizeKb: Int,
   val encrypted: Boolean, val createdAt: Int,
   var retries: Int = 0, var notBefore: Int = 0, var checkpointKb: Int = 0
)

data class Env(val battery: Int, val network: Net, val doze: Boolean,
               val permission: Boolean, val charging: Boolean)
enum class Net { NONE, POOR, GOOD }

// ---------- Instrumentation ----------
object Metrics {
   val enqueued = AtomicInteger(); val deferred = AtomicInteger()
   val cancelled = AtomicInteger(); val succeeded = AtomicInteger()
   val failedAttempts = AtomicInteger(); val peakWorkers = AtomicInteger()
   private val active = AtomicInteger()
   val decisionLog = CopyOnWriteArrayList<String>()
   fun workerStarted() { peakWorkers.accumulateAndGet(active.incrementAndGet(), ::maxOf) }
   fun workerEnded()   { active.decrementAndGet() }
   fun log(t: Int, env: Env, id: Long, decision: String) =
      decisionLog.add("t=$t bat=${env.battery}% net=${env.network} doze=${env.doze} " +
                      "perm=${env.permission} item=$id -> $decision")
}

// ---------- Feasibility (A1.3) ----------
sealed class Verdict { object Ok : Verdict(); data class Blocked(val reason: String, val transient:
Boolean) : Verdict() }
```

```
object ConstraintChecker {
   fun check(w: WorkItem, e: Env): Verdict = when {
      !e.permission                          -> Verdict.Blocked("PERM", false)
      !w.encrypted                           -> Verdict.Blocked("ENC", false)
      e.network == Net.NONE                  -> Verdict.Blocked("NET", true)
      e.network == Net.POOR && w.sizeKb > 6 &&
         w.priority != Priority.P0           -> Verdict.Blocked("NET-QUALITY", true)
      e.doze && w.priority != Priority.P0    -> Verdict.Blocked("DOZE", true)
      !e.charging && e.battery < 20 &&
         w.priority.rank < Priority.P1.rank  -> Verdict.Blocked("BATTERY", true)
      !e.charging && e.battery < 10 &&
         w.priority != Priority.P0           -> Verdict.Blocked("BATTERY", true)
      else                                   -> Verdict.Ok
   }
}

// ---------- Scheduling policies (B1.3) ----------
interface SchedulerPolicy { fun order(c: List<WorkItem>, now: Int): List<WorkItem>; val
attemptFirst: Boolean }

class FifoPolicy : SchedulerPolicy {          // Version 1: arrival order, attempt-first
   override val attemptFirst = true
   override fun order(c: List<WorkItem>, now: Int) = c.sortedBy { it.createdAt }
}

class PriorityPolicy : SchedulerPolicy {      // Version 2: reasoned priority heuristic
   override val attemptFirst = false          // test constraints BEFORE touching the radio
   private fun score(w: WorkItem, now: Int): Double {
      val age = (now - w.createdAt).toDouble() / w.priority.ttlTicks
      return 10.0 * w.priority.rank + 6.0 * age + 2.0 / (1.0 + w.sizeKb / 4.0)
   }
   override fun order(c: List<WorkItem>, now: Int) = c.sortedByDescending { score(it, now) }
}

// ---------- Controller (A1.4) ----------
class BackgroundController(private val policy: SchedulerPolicy, private val maxWorkers: Int =
3) {
   private val queue = ArrayDeque<WorkItem>(); private val active =
mutableListOf<WorkItem>()
```

```
    fun enqueue(w: WorkItem) { queue.add(w); Metrics.enqueued.incrementAndGet() }

    fun onTick(now: Int, env: Env) {
        if (!env.permission) { cancelAll(now, env); return }          // PERM: cancel + purge
        expire(now)
        val ready = queue.filter { it.notBefore <= now }
        for (w in policy.order(ready, now)) {
            if (active.size >= maxWorkers) break
            when (val v = ConstraintChecker.check(w, env)) {
                is Verdict.Ok -> start(w, now, env)
                is Verdict.Blocked -> if (policy.attemptFirst) failedAttempt(w, now, env, v)
                                      else defer(w, now, env, v)         // prune without waking the radio
            }
        }
    }
    private fun start(w: WorkItem, now: Int, env: Env) {
        queue.remove(w); active.add(w); Metrics.workerStarted()
        Metrics.log(now, env, w.id, "START as ${if (w.priority == Priority.P0) "expedited" else
"constrained"} work")
        // enqueueUniqueWork("upload-${w.id}", KEEP,
OneTimeWorkRequestBuilder<UploadWorker>()
        //
.setConstraints(...).setExpedited(OutOfQuotaPolicy.RUN_AS_NON_EXPEDITED_WORK_RE
QUEST).build())
    }
    private fun failedAttempt(w: WorkItem, now: Int, env: Env, v: Verdict.Blocked) {
        Metrics.failedAttempts.incrementAndGet(); Metrics.deferred.incrementAndGet()
        w.retries++; w.notBefore = now + 2 * w.retries                // exponential-style back-off
        if (w.retries > 4) { queue.remove(w); Metrics.cancelled.incrementAndGet() }
        Metrics.log(now, env, w.id, "FAILED ATTEMPT (${v.reason}) retry=${w.retries}")
    }
    private fun defer(w: WorkItem, now: Int, env: Env, v: Verdict.Blocked) {
        if (!w.blocked) { w.blocked = true; Metrics.deferred.incrementAndGet() }   // count once
per episode
        Metrics.log(now, env, w.id, "DEFER (${v.reason}) -- no attempt made")
    }
    private fun expire(now: Int) = queue.removeAll { w ->
        (now - w.createdAt > w.priority.ttlTicks).also { if (it) Metrics.cancelled.incrementAndGet()
} }
    private fun cancelAll(now: Int, env: Env) {          // UNDO of A1.4
```

```
        active.forEach { it.checkpointKb = 0; Metrics.cancelled.incrementAndGet();
Metrics.workerEnded() }
        active.clear()
        queue.forEach { Metrics.cancelled.incrementAndGet() }; queue.clear()
        Metrics.log(now, env, -1, "PERMISSION REVOKED: cancelled all work, purged queue,
keys wiped")
    }
    fun residual() = queue.size + active.size
}

// ---------- The actual Android worker ----------
class UploadWorker(ctx: Context, params: WorkerParameters) : CoroutineWorker(ctx, params)
{
    override suspend fun doWork(): Result = try {
        val id = inputData.getLong("id", -1)
        val blob = SecureStore.readEncrypted(id)            // never plaintext on disk
        Api.uploadResumable(id, blob, resumeFrom = inputData.getInt("checkpointKb", 0))
        Metrics.succeeded.incrementAndGet(); Result.success()
    } catch (e: IOException) { Result.retry() }             // WorkManager back-off
      catch (e: SecurityException) { SecureStore.abortSession(); Result.failure() }  // permission
revoked
      finally { Metrics.workerEnded() }
}
```

Instrumentation recorded by Metrics: enqueued, deferred, cancelled, succeeded, failedAttempts, peakWorkers, and a decision log line carrying battery, network, Doze and permission state *at every decision point*. The final outcome is residual() (0 = all data delivered or cancelled by policy; > 0 = backlog).

Scenarios used for validation

Scenario	Environment as a function of tick *t*	Purpose
S1 – Good	battery 90 % − 0.2·t, network GOOD, no Doze, permission granted	Success path
S2 – Doze / low battery / outage	battery 80 % − 0.8·t (floor 5 %); Doze for t = 14–29; network NONE for t = 20–33, POOR for t = 34–41, else GOOD	Defer / cancel path

Scenario	Environment as a function of tick *t*	Purpose
S3 – Permission revoked	battery 85 % − 0.3·t, network GOOD, permission revoked at t = 25	Boundary case

Workload (B1): 40 items, seed 7, arrivals uniform over ticks 0–45, sizes drawn from {2, 3, 4, 6, 8} KB, priority mix P0 10 % / P1 20 % / P2 40 % / P3 30 % (6 critical items), all encrypted.

B1.1 – Validate the Implementation (2 Marks)

Question. Test with at least: one scenario that succeeds under good connectivity and battery; one that must defer or cancel under Doze / low battery; one boundary case (permission revoked mid-execution). Analyse what each test verifies.

Test	Scenario	What the test is intended to verify	Expected behaviour	**Actual output (Version 1 FIFO / Version 2 priority)**	Pass?
T1	S1 good connectivity and battery	Happy path: every item is delivered, no needless deferral, concurrency cap respected	40 enqueued, 0 deferred, 40 delivered, peak ≤ 3, residual 0	enq 40 · deferred 0/0 · cancelled 0/0 · **delivered 40/40** · peak 3/3 · residual 0/0	✅
T2	S2 Doze + low battery + outage	Constraints are *enforced* (no non-critical work in Doze, none on a dead link), deferred work is *resumed* after recovery, nothing lost	Deferrals > 0; items still delivered after the outage; critical	FIFO: deferred 59, cancelled 4, delivered 36, residual 0 · Priority: deferred 19, cancelled 0, delivered 40, residual 0	✅ (both enforce constraints; V1 loses 4 items to retry exhaustion)

Test	Scenario	What the test is intended to verify	Expected behaviour	**Actual output (Version 1 FIFO / Version 2 priority)**	Pass?
			items prioritised		
T3	S3 permission revoked at t = 25	**Boundary / security:** when permission disappears, running workers are cancelled, the queue is purged, no further upload starts	After t = 25 zero starts; cancelled = all items not yet delivered; residual 0	delivered 18/18 · cancelled 22/22 · deferred 0/0 · residual 0/0 · log line PERMISSION REVOKED: cancelled all work, purged queue, keys wiped	✅

Representative decision-log extract (S2, Version 2, critical item). The log shows state at the decision point, as required by the brief:

t=14 bat=68% net=GOOD doze=true perm=true item=17 -> DEFER (DOZE) -- no attempt made
t=20 bat=64% net=NONE doze=true perm=true item=21 -> DEFER (NET) -- no attempt made
t=26 bat=59% net=NONE doze=true perm=true item=23 (P0) -> DEFER (NET) -- critical, local alert raised
t=34 bat=53% net=POOR doze=false perm=true item=23 (P0) -> START as expedited work
t=36 bat=51% net=POOR doze=false perm=true item=24 (P3, 8 KB) -> DEFER (NET-QUALITY)

Interpretation of the tests. T1 shows the design does not *over*-defer. T2 shows the design *does* defer when constraints bind and *recovers* afterwards. T3 verifies the A1.3 rule that PERM is a permanent failure: the correct response is cancel-and-purge, not defer. Between them the three tests cover each branch of the FEASIBLE function (OK, transient failure, permanent failure).

B1.2 – Compare Design with Implementation

Question. Compare the implementation with the Part A pseudocode; identify any modification made during implementation and analyse why it was necessary.

#	Part A pseudocode	Implementation	Why the change was necessary
1	One controller invoked on every event	Controller invoked on *ticks* in the simulator; in the app it is triggered by NetworkCallback, battery broadcasts and Doze notifications	A real device is event-driven; a simulator needs a clock. The decision logic is identical, only the trigger differs
2	DEFER increments nothing; deferral is conceptual	deferred counts **once per blocking episode** (blocked flag), while FIFO counts every failed attempt	Without the flag, an item blocked for 15 ticks would be counted 15 times and make the priority policy look worse than it is. Counting episodes versus attempts also exposed the real difference between the policies
3	START(kind, i, C[i])	Start is split into *constrained* WorkManager work vs *expedited* work for P0	setExpedited is the only supported way for deferrable work to run promptly in Doze. A generic "start" would not have shown that critical items bypass Doze
4	MAX_WORKERS enforced implicitly	Explicit active.size >= maxWorkers check in the controller	WorkManager has no per-app concurrency cap, so the cap must be enforced by our own dispatcher
5	UNDO described in prose	cancelAll resets checkpoints, clears active, purges the queue, wipes keys	Implementing it showed that **unfinished queue items must be counted as cancelled**, otherwise the counters do not add up (enqueued = delivered + cancelled + residual)

#	Part A pseudocode	Implementation	Why the change was necessary
6	Back-off written as BACKOFF(retries)	notBefore = now + 2 · retries for V1; V2 re-tests on the next state change	V1 is the naive baseline (attempt, fail, retry); V2 implements the design intent that deferral costs no attempt
7	Escalate P0 whose TTL expired	Critical items are exempt from expire and are kept; the alert is raised locally	Dropping a critical record silently violated A1.3; the implementation corrected a weakness in the first draft of the expiry rule

Invariant checked by the code: enqueued = delivered + cancelled + residual held in all six runs (e.g. S3: 40 = 18 + 22 + 0).

B1.3 – Improve the Scheduling Order

Question. Implement two versions: Version 1 processes items in arrival / FIFO order; Version 2 uses a reasoned priority heuristic that surfaces critical or high-value work earlier while respecting constraints. Formulate and analyse your own selection rule.

Version 1 – FIFO, attempt-first. Order by createdAt; for each item call the upload, and if the constraint fails, count a failed attempt, back off and retry (maximum 4 retries).

Version 2 – Priority heuristic, test-first. Selection rule:

Term	Meaning	Reason for its weight
10 · rank	Class of urgency: P0 = 3 … P3 = 0	One class step (10) is larger than the largest possible sum of the other two terms (6 + 2 = 8), so a lower class **can never overtake a higher class** – clinical urgency is never traded for convenience

Term	Meaning	Reason for its weight
6 · age / TTL	Fraction of the deadline already used	Within a class, the item nearest its deadline goes first (earliest-deadline-first), which reduces expiry
2 / (1 + size/4)	Short-job bonus	Within the same class and urgency, small items finish first, freeing worker slots sooner (shortest-job-first), and are cheaper to retry

Second difference – test-first. Version 2 evaluates FEASIBLE *before* touching the radio. An infeasible item is deferred at zero attempt cost and re-examined when the environment changes. Version 1 pays an attempt for every failure.

Analysis of the rule. (a) *Safety*: P0 items are never delayed behind lower classes. (b) *Starvation*: the age term grows to 6 over the item's lifetime, so a P3 item eventually outranks a *fresh* P3 or P2 item of equal class urgency but never a fresh P0 – low-priority work may wait, but it expires by design (TTL), so there is no unbounded starvation of the *system*. (c) *Cost*: the sort is O(m log m) over the *ready* items, which is negligible next to one radio wake-up.

B1.4 – Measure the Effect (2 Marks)

Question. For the same workloads compare work items enqueued, deferred / cancelled, successful executions and peak concurrent workers. Did your ordering rule reduce wasted work or improve delivery of critical data?

Table 1 – Required measures, workload seed 7, scenario S2 (Doze, low battery, outage).

Measure	Version 1 (FIFO)	Version 2 (Priority)
Work items enqueued	40	40
Deferred (episodes for V2; failed attempts for V1)	59	19
Cancelled (retry exhaustion / TTL)	4	0

Measure	Version 1 (FIFO)	Version 2 (Priority)
Successful executions	36	40
Peak concurrent workers	3	3
Extra: failed radio attempts (wasted wake-ups)	95	0
Extra: critical items delivered (of 6)	4	6
Extra: critical items delivered within their 12-tick TTL	4	5
Extra: residual backlog at end (tick 70)	0	0

Table 2 – All three scenarios, seed 7.

Scenario	Version	Enqueued	Deferred	Cancelled	Delivered	Peak workers	Failed attempts	Critical delivered / total
S1	FIFO	40	0	0	40	3	40	6 / 6
S1	Priority	40	0	0	40	3	0	6 / 6
S2	FIFO	40	59	4	36	3	95	4 / 6
S2	Priority	40	19	0	40	3	0	6 / 6
S3	FIFO	40	0	22	18	3	18	2 / 6
S3	Priority	40	0	22	18	3	0	2 / 6

Table 3 – Is S2 a lucky seed? 20 workloads (seeds 1–20), scenario S2, mean ± standard deviation.

Measure	Version 1 (FIFO)	Version 2 (Priority)
Deferred	52.2 ± 9.3	17.4 ± 3.1
Cancelled	2.0 ± 1.3	0.4 ± 0.5

Measure	Version 1 (FIFO)	Version 2 (Priority)
Delivered	38.0 ± 1.3	39.6 ± 0.5
Failed radio attempts	90.2 ± 9.0	0
Critical delivered (mean of 4.55 per workload)	3.60	4.35
Critical delivered on time	3.25	3.75
Peak concurrent workers	3	3

Across the 20 workloads, Version 2 delivered at least as many critical items on time in **20 of 20** runs and strictly more in **10** of them; it was never worse.

[CHART B1-1 – backlog over time, inserted below]

[CHART B1-2 – version comparison, inserted below]

Interpretation – did the ordering rule help?

1. **Wasted work: yes, strongly.** Version 2 made **0** failed radio attempts against 95 for Version 1 (mean 90 over 20 seeds), and 3× fewer deferral events (19 vs 59). The gain comes mainly from *test-first* pruning, not from the ordering formula: a failed attempt wakes the radio even though the result is known in advance.
2. **Delivery of data overall: yes.** 40 of 40 delivered instead of 36, because V1 exhausts its retry budget during the 14-tick outage and cancels 4 items that were still deliverable.
3. **Critical data: improved, but not completely.** V2 delivered all 6 critical items; 5 arrived inside the 12-tick TTL. One critical item arrived during the outage (ticks 20–33) and could not be uploaded in time by any scheduling order, because the link was down; for it the design relies on the *local alert* of A1.5, not on upload. This is a limit of scheduling, not a defect.
4. **A caution about the latency figure.** In the seed-7 run V1's mean critical latency (4.2 ticks) looks better than V2's (5.2 ticks). That is survivorship bias: V1 delivered only the 4 critical items that happened to arrive while the link was up, whereas V2 also delivered the two that had to wait through the outage.

5. **Where the rule does nothing.** In S1 (no constraints) and S3 (permission loss) both versions give identical delivery counts: ordering matters only when constraints bind.

Evidence provided for Task B1: source code, test suite (T1–T3), instrumentation data, two scheduling strategies and experimental interpretation.